



**UNIVERSIDADE FEDERAL DO TOCANTINS
CÂMPUS UNIVERSITÁRIO DE PALMAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

**HENRIQUE NORONHA
LAURO OLIVEIRA MOTA
JOÃO VICTOR MELO DO NASCIMENTO
HÁTILAN CAIO ALVES FONTES**

**ANÁLISE COMPARATIVA DE GERENCIAMENTO DE RECURSOS DO
SO: ABLETON LIVE VS. FL STUDIO**

PALMAS (TO)

2026

Henrique Noronha
Lauro Oliveira Mota
João Victor Melo do Nascimento
Hátilan Caio Alves Fontes

**Análise Comparativa de Gerenciamento de Recursos do SO: Ableton Live vs.
FL Studio**

Relatório de Sistemas Operacionais
apresentado à Universidade Federal do
Tocantins para obtenção do título de Bacharel
em Ciência da Computação.

PALMAS (TO)

2026

SUMÁRIO

1	INTRODUÇÃO	0
2	REFERENCIAL TEÓRICO	1
2.1	Gerenciamento de Processos e <i>Threads</i>	1
2.1.1	Escalonador do Windows NT	1
2.1.2	Tempo Real Crítico vs. Não Crítico	1
2.2	Interrupções e Latência de I/O em Tempo Real	1
2.2.1	<i>Callback</i> de Áudio e Tamanho de <i>Buffer</i>	2
2.2.2	ASIO vs. WDM/DirectSound	2
2.2.3	Algoritmos Formais de Escalonamento em Tempo Real	2
2.3	Gerenciamento de Memória e Paginação	3
2.3.1	<i>Eager</i> vs. <i>Lazy Allocation</i>	3
2.3.2	<i>Private Bytes</i>	3
3	METODOLOGIA	4
4	RESULTADOS	5
4.1	Testes 1 e 2: consumo em repouso e carga I/O-bound	5
4.2	Teste 3: escalonamento de threads	8
4.3	Teste 4: latência com driver nativo	9
4.4	Teste 5: estresse CPU-bound e ponto de ruptura	9
5	DISCUSSÃO	11
5.1	Análise do Teste 1 (repouso)	11
5.2	Análise do Teste 2 (20 trilhas AIFF)	11
5.3	Análise do Teste 3 (escalonamento de threads)	11
5.4	Análise do Teste 4 (latência)	11

5.5	Análise do Teste 5 (estresse CPU-bound)	12
6	CONCLUSÃO	14
	REFERÊNCIAS	15

1 INTRODUÇÃO

Os Sistemas Operacionais (SO) evoluíram bastante ao longo do tempo. Saíram de monitores simples de execução sequencial e se tornaram sistemas complexos, que precisam dar conta de hardware e software ao mesmo tempo. Enquanto as primeiras gerações se preocupavam basicamente com a abstração de comandos básicos e com a organização de arquivos, os sistemas atuais precisam lidar com uma arquitetura de múltiplos núcleos (*multicore*), memória virtual e interrupções em escala de milissegundos. Nesse cenário, muitos computadores pessoais acabam sendo usados como Estações de Trabalho de Áudio Digital (DAWs, do inglês *Digital Audio Workstations*). As DAWs são softwares complexos, e a performance delas não depende só da força bruta do processador, mas também de como o SO gerencia as *threads*, aloca memória e cuida das operações de entrada e saída (I/O) em alta velocidade.

O grande desafio desse tipo de aplicação é que ela precisa operar sob restrições de tempo real. Segundo a classificação de Silberschatz, Galvin e Gagne (2015), as DAWs se enquadram como sistemas de tempo real não crítico (*soft real-time*). Isso quer dizer que um atraso no atendimento de um processo não causa falha grave no computador, mas atrapalha bastante a qualidade do serviço. No caso do áudio, isso se manifesta como falhas e artefatos sonoros. Para evitar que isso aconteça, o SO precisa garantir que as *threads* de áudio tenham preferência sobre os processos comuns, reduzindo a latência de despacho e a latência de interrupção.

A partir disso, este trabalho faz uma análise comparativa do consumo e do gerenciamento de recursos do SO entre duas das DAWs mais usadas no mercado: o Ableton Live e o FL Studio. A ideia é mapear como cada arquitetura afeta o consumo de CPU, o comportamento *multithread*, o uso de memória RAM e a latência de I/O sob as mesmas condições de estresse. Com isso, esperamos mostrar na prática como as decisões de escalonamento e a arquitetura do SO impactam ambientes de alta exigência em tempo real.

2 REFERENCIAL TEÓRICO

2.1 Gerenciamento de Processos e *Threads*

Um processo é um programa em execução, dotado de espaço de endereçamento próprio e dos recursos a ele atribuídos pelo SO (SILBERSCHATZ; GALVIN; GAGNE, 2015). Dentro de um processo podem coexistir múltiplas *threads*: unidades menores de execução que compartilham o mesmo espaço de endereçamento, mas mantêm pilha e registradores próprios (TANENBAUM; BOS, 2016). Essa partilha torna a troca de contexto entre *threads* muito mais barata do que entre processos — razão pela qual motores de áudio recorrem ao *multithreading* para distribuir o processamento de blocos de amostras entre os núcleos do hardware.

2.1.1 Escalonador do Windows NT

O escalonador do Windows NT é preemptivo baseado em prioridades, com 32 níveis (0–31): a faixa de usuário ocupa os níveis 1–15 e a faixa de tempo real, 16–31 (STALLINGS, 2017). Os valores observados neste trabalho situam-se na faixa de usuário: *Normal* (8), *Above Normal* (10) e *High* (12–13). A cada *thread* o sistema atribui uma prioridade base (derivada da classe do processo) e, em tempo de execução, pode aplicar um *priority boosting* temporário — elevação automática para compensar *starvation* ou premiar *threads* recém-acordadas de I/O (STALLINGS, 2017). O *boost* decai gradualmente até retornar à base, o que explica a divergência entre prioridade base e dinâmica observável em ferramentas como o *Process Explorer*.

2.1.2 Tempo Real Crítico vs. Não Crítico

Silberschatz, Galvin e Gagne (2015) distinguem dois regimes de tempo real. Em sistemas de tempo real crítico (*hard real-time*), perder um prazo é uma falha absoluta — inaceitável em aviação ou em dispositivos médicos. Em sistemas de tempo real não crítico (*soft real-time*), a perda eventual de um prazo degrada a qualidade do serviço, mas não compromete o sistema. DAWs caem nesta segunda categoria: um *buffer* não preenchido a tempo produz *dropout* audível, mas a máquina continua estável. Basta, portanto, que as *threads* de áudio operem em prioridade superior à dos processos comuns para que o escalonador as despache primeiro sob contenção.

2.2 Interrupções e Latência de I/O em Tempo Real

Uma interrupção de hardware é um sinal assíncrono pelo qual um periférico avisa o processador de que requer atenção imediata (SILBERSCHATZ; GALVIN; GAGNE,

2015). Ao recebê-lo, o processador suspende a *thread* corrente, salva seu estado e transfere o controle para a Rotina de Serviço de Interrupção (ISR). O intervalo entre o sinal e o início da ISR é a latência de interrupção; somada ao tempo de troca de contexto até a *thread* destino, resulta na latência de despacho (STALLINGS, 2017). Em áudio, qualquer atraso de despacho que exceda o período do *buffer* se manifesta como silêncio ou artefato sonoro.

2.2.1 *Callback* de Áudio e Tamanho de *Buffer*

Drivers de áudio operam no modelo *pull callback*: o hardware solicita periodicamente um bloco de N amostras e o motor de áudio tem até o prazo do próximo bloco para entregá-lo (FONS, 2010). A relação entre *buffer* e latência é direta:

$$L = \frac{N}{f_s} \quad (1)$$

onde L é a latência em segundos, N o tamanho do bloco em amostras e f_s a taxa de amostragem. Por exemplo, com $f_s = 44.100$ Hz e $N = 256$, a latência fica em torno de 5,8 ms. Reduzir N diminui a latência percebida, mas aumenta a frequência das interrupções e, com isso, a carga sobre o escalonador (KIM; HAHN, 2003).

2.2.2 ASIO vs. WDM/DirectSound

O Windows expõe duas rotas de áudio ao aplicativo. Pelo WDM/DirectSound, o fluxo passa pelo *Kernel Audio Mixer* (KMixer), responsável por misturar áudio de múltiplos aplicativos antes de enviá-lo ao hardware (Microsoft Corporation, 2023a). Cada ciclo de *buffer* acorda o KMixer, acrescentando latência e interrupções extras. Já o protocolo ASIO (*Audio Stream Input/Output*) entrega ao driver do fabricante acesso direto ao hardware, ignorando o KMixer (Microsoft Corporation, 2023a). O ganho é uma redução substancial de latência e de carga sobre o escalonador.

Já o protocolo ASIO (*Audio Stream Input/Output*) dá ao driver do fabricante acesso direto ao hardware, pulando o KMixer (Microsoft Corporation, 2023a). O resultado é menos latência e menos pressão sobre o escalonador.

2.2.3 Algoritmos Formais de Escalonamento em Tempo Real

Vale lembrar que existe uma linha de pesquisa formal sobre escalonamento de prazos em tempo real. Liu e Layland (1973) formalizaram o problema com o algoritmo *Rate Monotonic*, e Buttazzo (2011) mostraram que garantir prazos formalmente exige análise de pior caso (WCET). As DAWs, por serem *soft real-time*, não usam esse rigor. Elas apenas aumentam a prioridade das *threads* de áudio no escalonador comum do SO, abrindo mão da garantia formal em troca de simplicidade de implementação.

2.3 Gerenciamento de Memória e Paginação

Cada processo recebe do SO um espaço de endereçamento virtual contíguo e isolado, independente da memória física disponível (SILBERSCHATZ; GALVIN; GAGNE, 2015). A tradução entre endereço virtual e físico é feita pela MMU com base nas tabelas de páginas do *kernel*. Na paginação por demanda, uma página só é mapeada a um quadro físico quando acessada pela primeira vez — evento chamado de falta de página (*page fault*) (TANENBAUM; BOS, 2016). Resolvê-la exige suspender a *thread*, localizar (ou ler do disco) a página e retomar a execução: uma latência não determinística que pode variar de microssegundos a milissegundos.

Para áudio em tempo real, uma falta de página no meio do processamento de um bloco é especialmente cara: a *thread* fica bloqueada esperando o *kernel* resolver, enquanto o prazo do *buffer* continua correndo (KIM; HAHN, 2003). Sistemas críticos resolvem esse problema fixando páginas na RAM com `mlock()` no Linux ou `VirtualLock()` no Windows. As DAWs comerciais costumam adotar uma estratégia diferente: alocam boa parte da memória logo na inicialização, o que tem efeito parecido na prática.

2.3.1 *Eager* vs. *Lazy Allocation*

Tanenbaum e Bos (2016) contrastam dois modelos de alocação:

- *Eager allocation*: reserva e inicializa toda a memória prevista logo na inicialização do processo. Elimina faltas de página no caminho crítico, ao custo de uma pegada elevada em repouso.
- *Lazy allocation*: aloca recursos só quando demandados. Mantém pegada mínima em repouso, mas introduz latência não determinística no momento da primeira alocação.

2.3.2 *Private Bytes*

O contador *Private Bytes* do Windows mede a memória virtual comprometida exclusivamente pelo processo, sem contar DLLs e outras regiões compartilhadas (STALLINGS, 2018). Essa é a métrica que melhor reflete o custo real de memória da aplicação, já que mostra o que precisaria ser paginado para disco caso o sistema ficasse sem RAM física. No caso das DAWs, o crescimento desse contador ao carregar trilhas vem principalmente dos *buffers* de pré-leitura e das estruturas de decodificação que precisam ficar em memória para garantir reprodução de baixa latência.

3 METODOLOGIA

O estudo adota uma abordagem quantitativa e comparativa para analisar o gerenciamento de recursos de *Digital Audio Workstations* (DAWs) sob a ótica de Sistemas Operacionais. O foco é a interação entre o *kernel* do Windows 11 (build 26200, x64) e os motores de áudio do Ableton Live 12 e do FL Studio 24. Os experimentos foram feitos em um notebook ASUS Vivobook K3605ZF com processador Intel Core i5-12500H (12 núcleos, 16 *threads*, base 3,1 GHz), 16 GB de RAM DDR4 e GPU NVIDIA (driver `nvlldmkm.sys` v581.29). Para preservar as escolhas arquiteturais nativas de cada fabricante, o FL Studio rodou com o driver FL Studio ASIO e o Ableton Live rodou com MME/DirectX. O *Fruity Limiter* do canal Master do FL Studio foi removido antes dos testes para manter paridade na cadeia de sinal. O Windows Defender ficou ativo em todas as sessões, em condições iguais para as duas DAWs.

A coleta de dados foi feita com três ferramentas complementares. O Performance Monitor (PerfMon) capturou contadores do *kernel* a 1 Hz, especificamente **% Tempo de Processador**, **Private Bytes** e **Contagem de Threads** por processo, e os dados foram exportados via `relog` para o formato `.csv` (Microsoft Corporation, 2023b). O Process Explorer (Sysinternals) foi usado para inspecionar as *threads* individuais, suas prioridades de escalonamento e o consumo de CPU por unidade de execução. O LatencyMon v7.31 (BERNSEE, 2023) mediu a latência *interrupt-to-process*, os tempos de rotinas DPC e ISR e a contagem de *hard pagefaults* no nível do sistema.

O protocolo experimental foi composto por cinco testes sequenciais. O Teste 1, de repouso, coletou dados durante 2,5 minutos com a DAW aberta e sem projeto carregado. O Teste 2, de carga *I/O-bound*, adicionou a reprodução simultânea em *loop* de 20 trilhas AIFF (PCM não comprimido) por 3 minutos. No Teste 3, de escalonamento, o Process Explorer foi usado para inspecionar a composição, a prioridade e a distribuição das *threads* com o projeto ativo. No Teste 4, de latência, o LatencyMon foi executado por 2 min 50 s sobre o mesmo projeto, registrando DPCs, ISRs e *pagefaults*. Por fim, no Teste 5, de estresse CPU-bound, foram inseridas cinco instâncias do reverb nativo de cada DAW no canal Master, e o tamanho do *buffer* foi reduzido aos poucos de 4096 até 128 *samples*. O ponto de ruptura (*buffer underrun*) foi identificado pelo aparecimento de *dropouts* audíveis.

4 RESULTADOS

4.1 Testes 1 e 2: consumo em repouso e carga I/O-bound

Os Testes 1 e 2 avaliaram o comportamento de CPU, memória e *threads* em dois cenários consecutivos: DAW em repouso (sem projeto) e com reprodução simultânea de 20 trilhas AIFF em *loop*. Os dados, amostrados a 1 Hz via PerfMon durante 2,5 e 3 minutos respectivamente, estão consolidados nas Tabelas 1 e ??.

Tabela 1 – Resumo estatístico do Teste 1 (repouso) e do Teste 2 (20 trilhas AIFF).

Teste	Métrica	DAW	Mín.	Média	Máx.
T1: repouso	% CPU	Ableton Live 12	0,00%	10,95%	33,73%
		FL Studio (FL64)	0,00%	12,38%	37,72%
	Private Bytes (MB)	Ableton Live 12	338,6	536,8	538,3
		FL Studio (FL64)	95,6	95,8	95,8
	Threads ativas	Ableton Live 12	8	54,8	56
		FL Studio (FL64)	44	44,0	45
T2: 20 trilhas	% CPU	Ableton Live 12	0,00%	3,74%	20,11%
		FL Studio (FL64)	0,00%	4,80%	31,16%
	Private Bytes (MB)	Ableton Live 12	601,6	601,7	604,5
		FL Studio (FL64)	137,6	147,6	147,6
	Threads ativas	Ableton Live 12	54	55,3	56
		FL Studio (FL64)	26	27,0	28

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

Em repouso, o FL Studio consumiu CPU média um pouco maior que o Ableton Live (12,38% contra 10,95%). Por outro lado, o Ableton Live alocou 5,6 vezes mais memória (536,8 MB contra 95,8 MB) e manteve quase o dobro de *threads* ativas (54,8 contra 44,0). Com as 20 trilhas em reprodução, as duas DAWs reduziram o consumo médio de CPU: o Ableton caiu de 10,95% para 3,74% e o FL Studio caiu de 12,38% para 4,80%. A memória cresceu de forma moderada, +64,9 MB no Ableton e +51,8 MB no FL Studio. O FL Studio reduziu suas *threads* de 44 para 27 ao carregar o projeto, uma queda de 38%, enquanto o Ableton praticamente não mudou (54,8 para 55,3). As Figuras 1 até 5 mostram esses comportamentos.

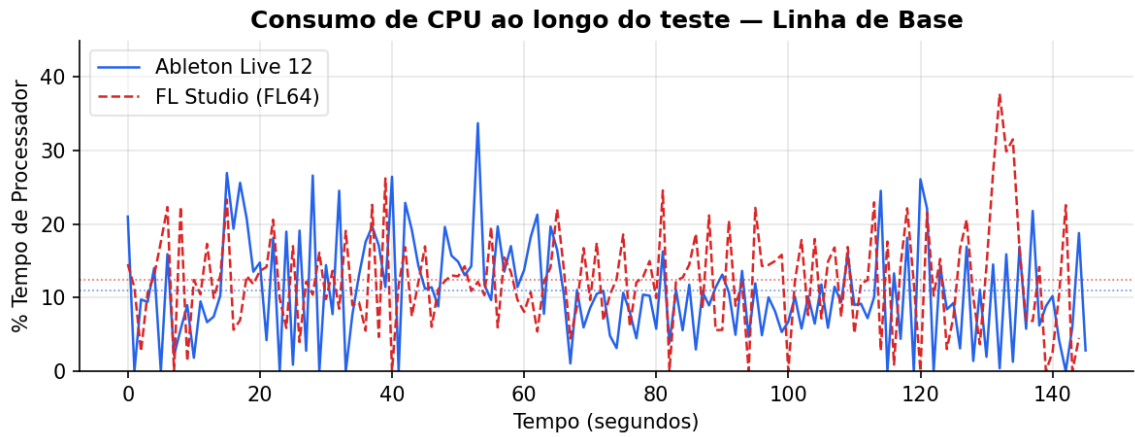


Figura 1 – Consumo de CPU em repouso no Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

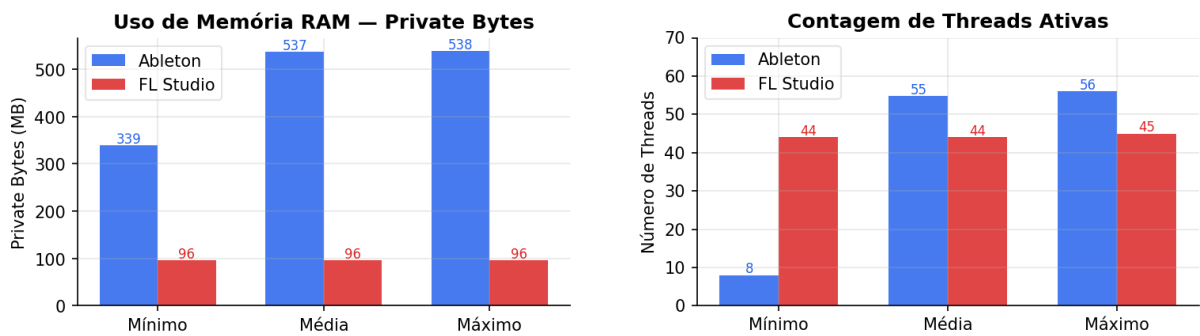


Figura 2 – Uso de RAM (*Private Bytes*) e número de *threads* em repouso no Teste 1.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

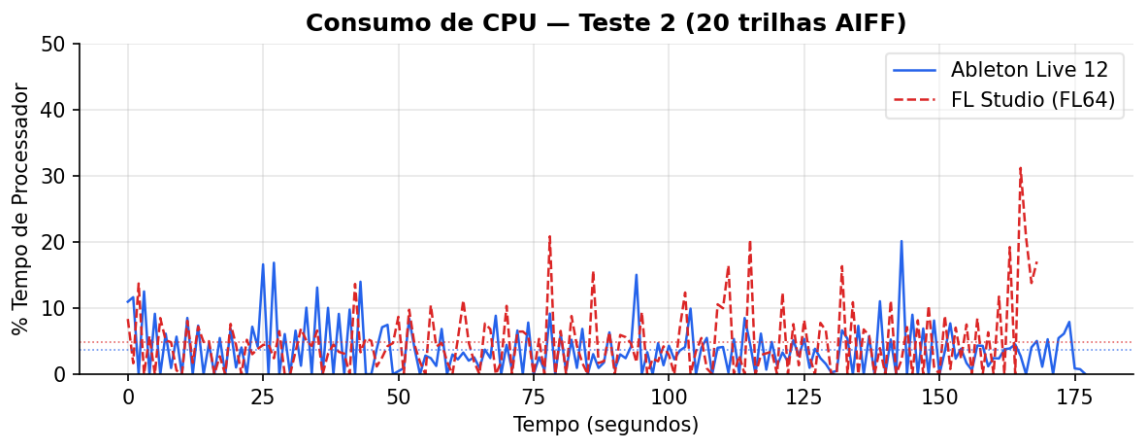


Figura 3 – Consumo de CPU durante a reprodução de 20 trilhas AIFF no Teste 2.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

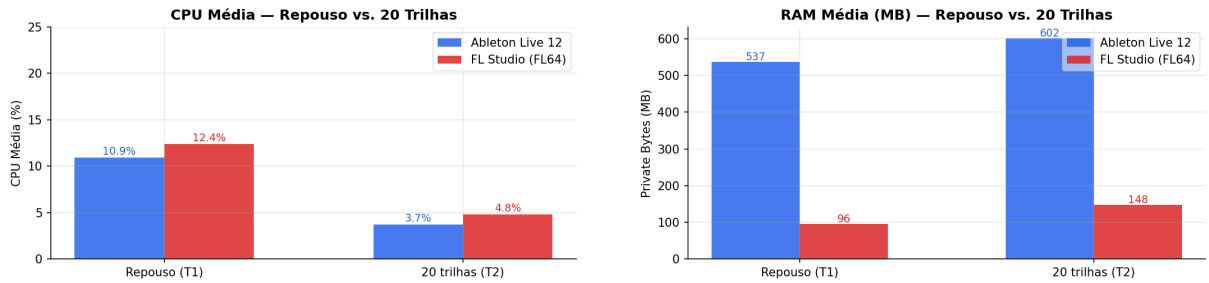


Figura 4 – Comparação de CPU e RAM médias entre repouso (T1) e 20 trilhas (T2).

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

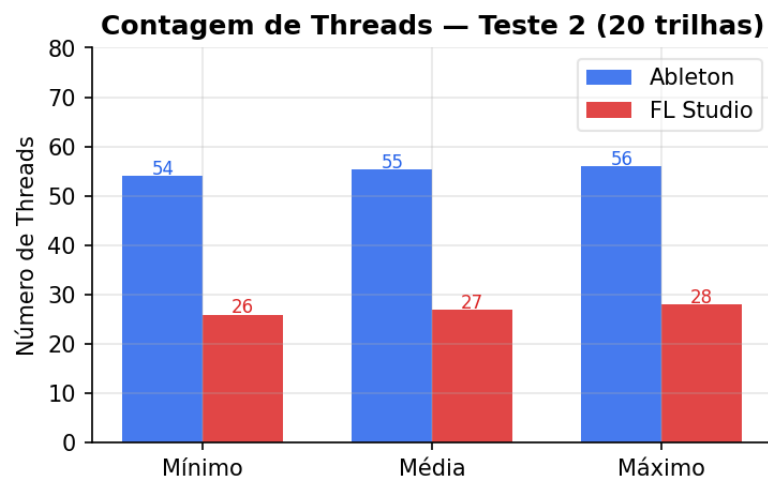


Figura 5 – Número de *threads* ativas com 20 trilhas em reprodução no Teste 2.

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2 Teste 3: escalonamento de threads

A inspeção via Process Explorer com o projeto em reprodução mostrou diferenças relevantes na composição e na prioridade das *threads* (Tabela 2). O FL Studio opera com prioridade base 10 (*Above Normal*) e dinâmica 12 (*High*), enquanto o Ableton Live opera com base 8 (*Normal*) e dinâmica 10. Nenhuma das duas atingiu o nível *Time Critical* (15). Quanto à composição, 14 das 29 *threads* do FL Studio pertencem ao módulo QuickFontCache, que cuida da renderização da interface, e só 5 estão associadas ao FLEngine_x64, que é o motor de áudio. No Ableton Live, cerca de 30 das 55 *threads* vêm do ucrtbase.dll, que é o *thread pool* do C runtime. As Figuras 6 e 7 detalham esses dados.

Tabela 2 – Composição e prioridade das *threads* no Teste 3, obtidas via Process Explorer.

Parâmetro	FL Studio (FL64)	Ableton Live 12
Total de <i>threads</i>	29	55
Prioridade base (thread principal)	10 (<i>Above Normal</i>)	8 (<i>Normal</i>)
Prioridade dinâmica	12 (<i>High</i>)	10 (<i>Above Normal</i>)
CPU da thread principal	0,93%	0,75%
Thread <i>Time Critical</i> (15)?	Não	Não
Módulo dominante	QuickFontCache	ucrtbase
Thread de driver de áudio	ilwasapi2asio_x64	DSOUND.dll

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

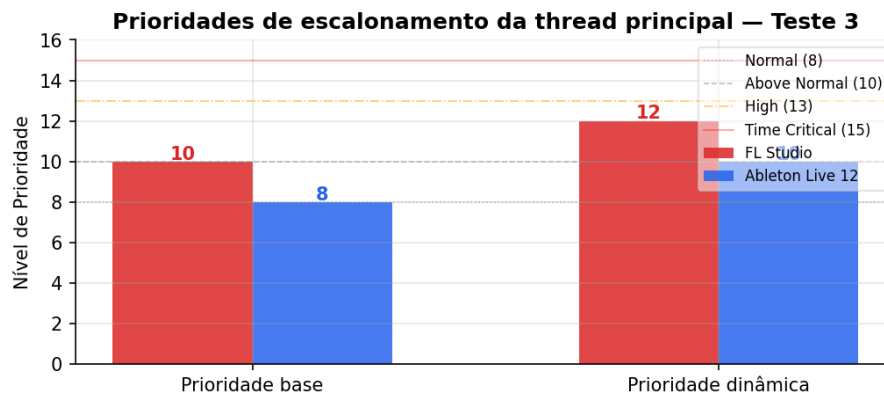


Figura 6 – Prioridades de escalonamento da *thread* principal no Teste 3.

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

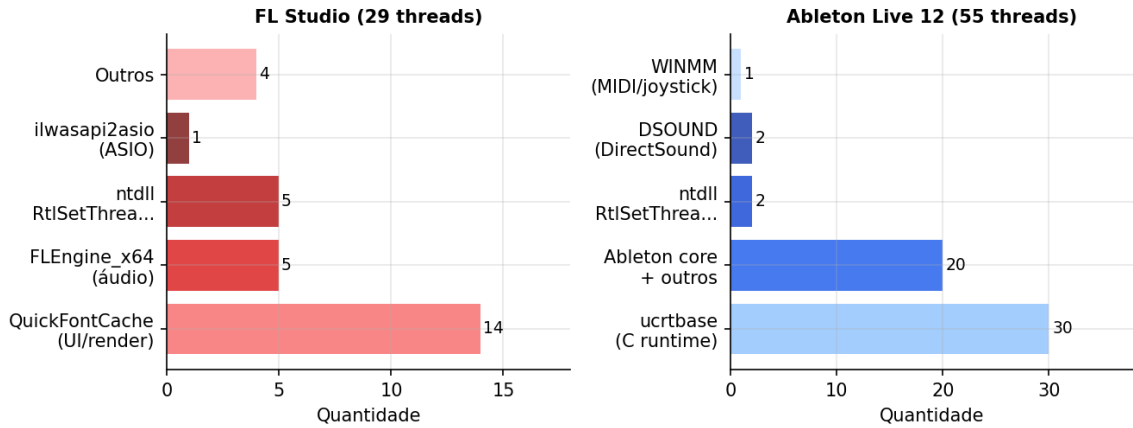


Figura 7 – Distribuição das *threads* por módulo no Teste 3.

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

4.3 Teste 4: latência com driver nativo

O LatencyMon classificou o sistema como apto nas duas DAWs (Tabela 3). A latência máxima *interrupt-to-process* foi de 805,40 μ s no Ableton Live e 939,80 μ s no FL Studio, ambas dentro do limite aceitável de 1000 μ s. O maior DPC do FL Studio veio do driver NVIDIA `nvlddmkm.sys` (647,07 μ s). Em ambas as sessões, o processo com mais *hard pagefaults* foi o `msmpeng.exe` do Windows Defender. O FL Studio teve percentual de CPU em DPCs um pouco maior (0,090% contra 0,074%) e em ISRs (0,044% contra 0,023%).

Tabela 3 – Resultados de latência do Teste 4, obtidos com o LatencyMon v7.31.

Métrica	Ableton Live 12	FL Studio (FL64)
Conclusão do sistema	Apto	Apto
Latência máx. int.-to-proc. (μ s)	805,40	939,80
Latência méd. int.-to-proc. (μ s)	8,50	16,96
DPC máximo (μ s)	449,81	647,07
ISR máximo (μ s)	267,52	401,24
Tempo total em DPCs (%)	0,074	0,090
Tempo total em ISRs (%)	0,023	0,044
<i>Hard pagefaults</i> totais	306	378
Driver com maior DPC	<code>tcpip.sys</code>	<code>nvlddmkm.sys</code>

Fonte: dados coletados pelos autores via LatencyMon v7.31, 26 maio 2026.

4.4 Teste 5: estresse CPU-bound e ponto de ruptura

Com cinco instâncias de reverb ativas no canal Master e as 20 trilhas em reprodução, o tamanho do *buffer* foi reduzido aos poucos até começar a aparecer *dropouts* audíveis (Tabela 4). O FL Studio (ASIO) se manteve estável até 256 *samples* (6 ms) e quebrou

em 128 *samples*. O Ableton Live (MME/DirectX) começou a apresentar problemas já em 1024 *samples* (23,2 ms), com degradação cada vez maior nos valores menores. As Figuras 8 e 9 resumem esse comportamento.

Tabela 4 – Ponto de ruptura observado no Teste 5 (estresse CPU-bound).

Buffer (smp)	Latência (ms)	FL Studio (ASIO)	Ableton (MME/DirectX)
4096	92,9	—	Estável
2048	46,4	—	Estável
1024	23,2	—	Glitch
512	11,6	Estável	Glitch piora
384	9,0	Estável	—
256	5,8	Mínimo estável	Glitch forte
128	2,9	Glitch	—

Fonte: dados coletados pelos autores, 27 maio 2026.

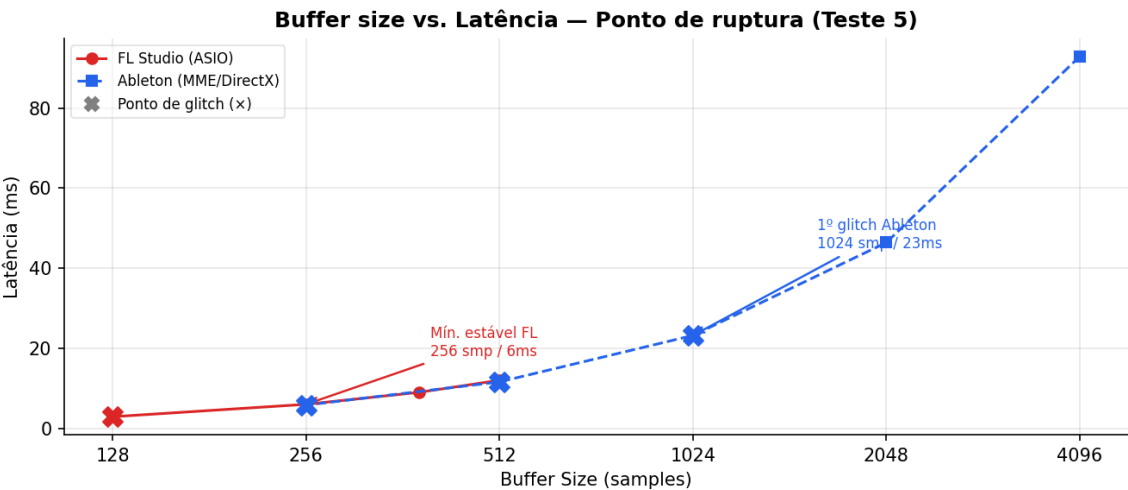


Figura 8 – Tamanho do *buffer* e latência correspondente no Teste 5. O marcador $\times$ indica o ponto de *dropout* em cada DAW.
Fonte: dados coletados pelos autores, 27 maio 2026.

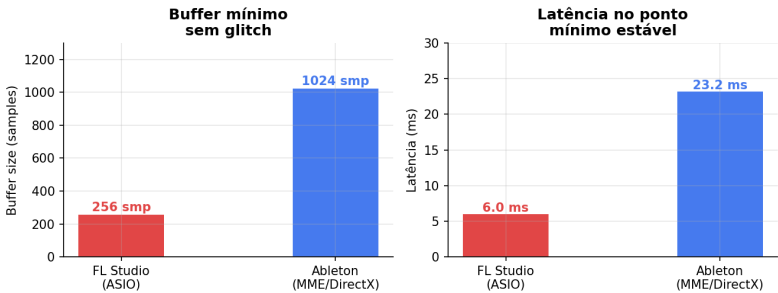


Figura 9 – Buffer mínimo estável e latência correspondente no Teste 5.
Fonte: dados coletados pelos autores, 27 maio 2026.

5 DISCUSSÃO

5.1 Análise do Teste 1 (repouso)

Os resultados do Teste 1 mostram que as duas DAWs partem de pontos de inicialização bem diferentes. O Ableton Live segue a estratégia de *eager allocation*: aloca memória e cria *threads* logo na inicialização (cerca de 537 MB e 55 *threads*), o que mantém a latência de despacho baixa desde o início (SILBERSCHATZ; GALVIN; GAGNE, 2015). O FL Studio adota *lazy allocation* e mantém só o essencial em repouso (cerca de 96 MB e 44 *threads*), deixando para alocar mais recursos só quando o projeto for carregado (TANENBAUM; BOS, 2016).

5.2 Análise do Teste 2 (20 trilhas AIFF)

O Teste 2 trouxe três pontos interessantes. Primeiro, a redução de CPU sob carga *I/O-bound* nas duas DAWs pode ser explicada pelo driver de áudio nativo: quando ele assume o controle exclusivo do hardware, acaba eliminando a sobrecarga do *Kernel Mixer* do Windows. Segundo, o crescimento de RAM foi parecido em termos absolutos (+65 MB no Ableton e +52 MB no FL Studio), o que indica que o custo por trilha AIFF é parecido nas duas arquiteturas. Terceiro, a queda de *threads* no FL Studio (de 44 para 27) sugere que o motor de áudio consolida *threads* genéricas em outras mais especializadas quando o projeto é carregado (STALLINGS, 2017).

5.3 Análise do Teste 3 (escalonamento de threads)

O FL Studio opera com prioridade mais alta (base 10 e dinâmica 12), o que em tese deveria garantir maior fatia de CPU em situações de disputa (SILBERSCHATZ; GALVIN; GAGNE, 2015). Mas, das 29 *threads* totais, 14 pertencem ao **QuickFontCache**, ou seja, parte desse orçamento de CPU é consumida pela interface gráfica, e não pelo motor de áudio. O Ableton Live, mesmo com prioridade base menor (8), distribui sua carga em 55 *threads* usando o *thread pool* do **ucrtbase**, o que favorece a escalabilidade nos 16 *threads* lógicos do i5-12500H (TANENBAUM; BOS, 2016). Vale notar que nenhuma das duas DAWs usa prioridade *Time Critical* (15), o que confirma o enquadramento delas como sistemas *soft real-time* (STALLINGS, 2017).

5.4 Análise do Teste 4 (latência)

O Teste 4 foi feito em um ambiente controlado, com o navegador e outros aplicativos secundários fechados. O LatencyMon classificou as duas DAWs como aptas para operação

de áudio em tempo real. O Ableton Live teve latência máxima de $805\ \mu\text{s}$ e o FL Studio de $939\ \mu\text{s}$, uma diferença de cerca de 17%, com ambas ficando abaixo do limite crítico de $1000\ \mu\text{s}$ definido pelo LatencyMon.

O principal fator de latência no FL Studio foi o driver NVIDIA `nvlddmkm.sys`, que gerou o DPC máximo de $647\ \mu\text{s}$. Isso indica que a GPU do sistema gera interrupções de duração mais alta e acaba disputando o escalonador com o motor de áudio, independentemente da DAW em uso (SILBERSCHATZ; GALVIN; GAGNE, 2015). No Ableton Live, o maior DPC veio do `tcpip.sys`, o que sugere que o subsistema gráfico estava pressionando menos o escalonador durante essa sessão específica.

Esse resultado contrasta com uma sessão anterior do FL Studio que foi feita com o Windows Defender em surto de varredura. Nessa sessão a latência chegou a $5274\ \mu\text{s}$ e a conclusão do LatencyMon foi de não apto. A comparação entre as duas sessões mostra que interferências de processos externos, principalmente antivírus e drivers de GPU, podem ter impacto determinante na estabilidade de áudio em tempo real, às vezes até maior do que as próprias diferenças entre as arquiteturas das DAWs (TANENBAUM; BOS, 2016). Em condições equivalentes, as duas aplicações parecem igualmente capazes de operar sem *dropouts*.

5.5 Análise do Teste 5 (estresse CPU-bound)

O resultado do Teste 5 precisa ser lido com algum cuidado, porque as duas DAWs usaram drivers de áudio diferentes. Essa diferença em si já é um dado interessante sobre as escolhas arquiteturais de cada fabricante.

O FL Studio com driver ASIO se manteve estável até 256 *samples* (cerca de 6 ms), o que confirma o benefício do acesso direto ao hardware oferecido pelo ASIO: como ele não passa pelo *Kernel Audio Mixer* (KMixer) do Windows, acaba reduzindo o número de interrupções por ciclo de *buffer* e diminuindo a carga sobre o escalonador (Microsoft Corporation, 2023a). Esse comportamento é coerente com a prioridade de *thread* mais alta (base 10 e dinâmica 12) observada no Teste 3, que dá ao motor de áudio uma janela maior para preencher cada bloco antes do prazo.

Já o Ableton Live com MME/DirectX começou a apresentar *dropouts* já em 1024 *samples* (23,2 ms), quatro vezes acima do ponto de ruptura do FL Studio. Isso era esperado pelo que foi visto no referencial teórico: o caminho WDM/DirectX passa pelo KMixer, que adiciona latência e interrupções extras a cada ciclo de *buffer* (Microsoft Corporation, 2023a; KIM; HAHN, 2003). Sob carga CPU-bound (cinco reverbs em convolução ao mesmo tempo), esse custo extra torna inviável manter buffers menores sem *underruns*.

É importante ressaltar que essa comparação direta dos pontos de ruptura reflete sobretudo a diferença entre os modelos de driver, e não necessariamente a capacidade do motor de áudio de cada DAW em si. Num cenário em que as duas DAWs usassem ASIO, o

resultado provavelmente seria diferente. Essa é uma limitação do nosso estudo e também uma direção para trabalhos futuros.

6 CONCLUSÃO

Este estudo mostrou que, do ponto de vista do gerenciamento de recursos do Sistema Operacional, o Ableton Live e o FL Studio seguem abordagens arquiteturais bem diferentes. Em resumo, os resultados mostram que o Ableton Live prioriza a alocação antecipada (*eager allocation*): ele reserva o espaço necessário e cria um *pool* grande de *threads* logo no início, o que garante baixa latência de despacho e ajuda a dividir o trabalho entre os 16 *threads* lógicos do hardware. Já o FL Studio funciona com alocação sob demanda (*lazy allocation*), mantendo só as estruturas mínimas até o momento em que a carga de trabalho chega.

Também observamos que o FL Studio tenta compensar isso atribuindo uma prioridade de escalonamento mais alta (nível dinâmico 12) para a sua *thread* principal. Só que boa parte desse esforço computacional não acaba indo para o motor de áudio, e sim para a renderização da interface gráfica (o módulo `QuickFontCache`). No teste de latência, feito em ambiente controlado, as duas DAWs foram classificadas como aptas: o Ableton Live com $805\mu\text{s}$ e o FL Studio com $939\mu\text{s}$ de latência máxima *interrupt-to-process*, ambas abaixo do limite de $1000\mu\text{s}$. O driver NVIDIA (`nvlddmkm.sys`) foi o principal gerador de DPCs altos no FL Studio, e o Windows Defender (`msmpeng.exe`) gerou *hard pagefaults* nos dois casos. Vale notar que uma sessão anterior do FL Studio com o Defender em surto de varredura chegou a $5274\mu\text{s}$, o que mostra que interferências externas podem ter um impacto até maior do que as próprias diferenças arquiteturais entre as DAWs.

Por fim, vimos que o uso de drivers nativos ajuda a reduzir a sobrecarga de CPU porque elimina a intervenção do *Kernel Mixer* do Windows. As duas DAWs operam como sistemas de tempo real não crítico (*soft real-time*) e nenhuma das duas chega a usar prioridade *Time Critical*.

Do ponto de vista prático, os resultados dão algumas pistas para dimensionar hardware e software. Para quem usa Ableton Live, máquinas com bastante RAM e processadores com muitos núcleos tendem a ser mais vantajosas. Já para quem usa FL Studio, um processador com bom desempenho *single-core* pode compensar a concentração de carga na *thread* principal. Do lado da engenharia de software, os dados sugerem que vale a pena isolar melhor as *threads* de áudio das rotinas de interface gráfica, para que o escalonador do SO não acabe penalizando dados sensíveis ao tempo em favor da renderização visual.

REFERÊNCIAS

- BERNSEE, M. **ASIO4ALL: Universal ASIO Driver for WDM Audio**. [S.l.], 2023. Disponível em: <<https://www.asio4all.org>>. Acesso em: 26 maio 2026.
- BUTTAZZO, G. C. **Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications**. 3. ed. New York: Springer, 2011.
- FONS, J. Real-time audio processing on general purpose operating systems. In: **Linux Audio Conference**. Utrecht: [s.n.], 2010.
- KIM, Y.; HAHN, M. Improving real-time audio performance under general purpose operating systems. **IEICE Transactions on Information and Systems**, E86-D, n. 1, p. 192–196, 2003.
- LIU, C. L.; LAYLAND, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. **Journal of the ACM**, v. 20, n. 1, p. 46–61, 1973.
- Microsoft Corporation. **Audio Drivers for Windows: ASIO Architecture**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-hardware/drivers/audio>>. Acesso em: 26 maio 2026.
- Microsoft Corporation. **Windows Performance Monitor: Technical Reference**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/perfmon>>. Acesso em: 26 maio 2026.
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. Tradução de: Operating System Concepts, 9th ed.
- STALLINGS, W. **Operating Systems: Internals and Design Principles**. 9. ed. Hoboken: Pearson, 2017.
- STALLINGS, W. **Arquitetura e Organização de Computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2018. Tradução de: Computer Organization and Architecture, 10th ed.
- TANENBAUM, A. S.; BOS, H. **Sistemas Operacionais Modernos**. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Tradução de: Modern Operating Systems, 4th ed.